Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date:**	**Enrolment No:92301733012**

Aim: To recommend an item to the user using content-based filtering approach

IDE: Google Colab

Theory:

There are a lot of applications where websites collect data from their users and use that data to predict the likes and dislikes of their users. This allows them to recommend the content that they like. Recommender systems are a way of suggesting or similar items and ideas to a user's specific way of thinking.

Recommender System is different types:

- **Collaborative Filtering:** Collaborative Filtering recommends items based on similarity measures between users and/or items. The basic assumption behind the algorithm is that users with similar interests have common preferences.
- **Content-Based Recommendation:** It is supervised machine learning used to induce a classifier to discriminate between interesting and uninteresting items for the user.

Content-Based Recommendation System: Content-Based systems recommends items to the customer similar to previously high-rated items by the customer. It uses the features and properties of the item. From these properties, it can calculate the similarity between the items.
In a content-based recommendation system, first, we need to create a profile for each item, which represents the properties of those items. From the user profiles are inferred for a particular user. We use these user profiles to recommend the items to the users from the catalog.

Item profile:
In a content-based recommendation system, we need to build a profile for each item, which contains the important properties of each item. For Example, If the movie is an item, then its actors, director, release year, and genre are its important properties, and for the document, the important property is the type of content and set of important words in it.
Let's have a look at how to create an item profile. First, we need to perform the TF-IDF vectorizer, here TF (term frequency) of a word is the number of times it appears in a document and The IDF (inverse document frequency) of a word is the measure of how significant that term is in the whole corpus.

User profile:
The user profile is a vector that describes the user preference. During the creation of the user's profile, we use a utility matrix that describes the relationship between user and item. From this information, the best estimate we can decide which item the user likes, is some aggregation of the profiles of those items.
Advantages and Disadvantages:

- **Advantages:**
    - No need for data on other users when applying to similar users.
    - Able to recommend to users with unique tastes.
    - Able to recommend new & popular items

- Explanations for recommended items.
- **Disadvantages**:
  - Finding the appropriate feature is hard.
  - Doesn't recommend items outside the user profile.

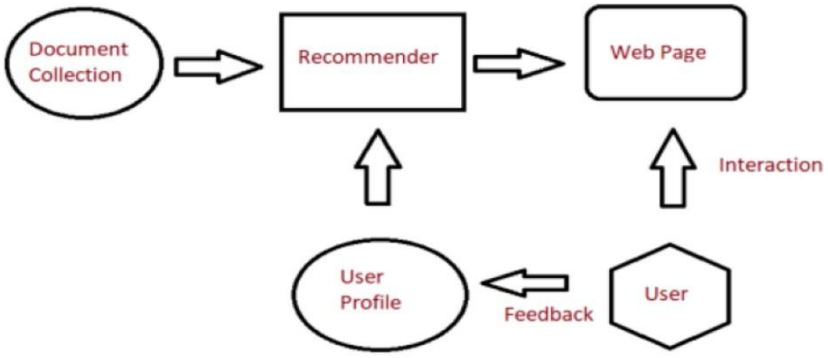

Fig: Recommender System

Pre Lab Exercise:

1. When can you use content-based recommendation system approach?

Content-based recommendation is used when item features and user preferences are available, and recommendations need to be personalized without relying on other users' data.

2.Write advantages of content-based filtering approach

- ✓ Provides **personalized recommendations** based on user preferences.
- ✓ **No need for other users' data** (works independently).
- ✓ Handles **new items (no cold start for items)** effectively.
- ✓ Easy to **understand and explain** recommendations.
- ✓ Preserves **user privacy** (no sharing of user data).
- ✓ Works well when **item features are rich and descriptive**.

3.Write disadvantages of content-based filtering approach.

- ✓ **Limited recommendations** (only similar items are suggested).
- ✓ **Lack of diversity** (no new or different items).
- ✓ **Cold start for new users** (no history → no recommendations).
- ✓ Depends heavily on **quality of item features**.
- ✓ Requires **feature engineering**, which can be complex.
- ✓ Cannot capture **changing user preferences easily**.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date:**	**Enrolment No:92301733012**

Program (Code):

```
!pip install pandas scikit-learn

import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from p
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (20
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-lear
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-lea
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from sci
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil
```

```
data = {
    'title': ['Movie A', 'Movie B', 'Movie C', 'Movie D'],
    'description': [
        'action adventure hero fight',
        'romantic love story relationship',
        'action thriller spy mission',
        'comedy fun laughter family'
    ]
}
```

[3] 0s
```
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(df['description'])
```

[4] 0s
```
similarity = cosine_similarity(tfidf_matrix)
```

[5] 0s
```
def recommend(movie_name):
    idx = df[df['title'] == movie_name].index[0]
    scores = list(enumerate(similarity[idx]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)

    print(f"Recommendations for {movie_name}:")
    for i in scores[1:3]:
        print(df.iloc[i[0]]['title'])
```

[6] 0s
```
recommend('Movie A')
```

```
Recommendations for Movie A:
Movie C
Movie B
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date:**	**Enrolment No:92301733012**

Results:

```
Recommendations for Movie A:
Movie C
Movie B
```

Observation:

- ✓ The dataset consists of items (movies) with textual descriptions, which act as the primary feature for generating recommendations.
- ✓ The TF-IDF (Term Frequency–Inverse Document Frequency) technique is used to convert textual data into numerical vectors, capturing the importance of words in each description.
- ✓ The cosine similarity measure is applied to compute similarity between items based on their TF-IDF vectors.
- ✓ Items with similar keywords (e.g., "action", "thriller") show higher similarity scores and are more likely to be recommended together.
- ✓ The recommendation function successfully identifies and ranks similar items based on similarity scores in descending order.
- ✓ The system does not require data from other users, as it relies only on item features (content-based approach). The quality of recommendations depends heavily on the richness and accuracy of item descriptions.
- ✓ The model performs well for items with clear and descriptive features but may give less accurate results when descriptions are too short or ambiguous.
- ✓ The system cannot recommend items outside the scope of the existing features, limiting diversity in recommendations.

Post Lab Exercise:

1. What changes you feel can be done, to improve the system that you have designed, as a part of in-house lab exercise.
   - Use more features like genre, ratings, and keywords.
   - Apply better text preprocessing (remove noise, use stemming).
   - Use advanced models like Word2Vec or BERT.
   - Try different similarity measures.

- Handle new users with initial preferences.
- Combine with collaborative filtering (hybrid system).
- Use a larger dataset.
- Add user feedback for better recommendations.
- Rank results using popularity or ratings.

2. Recommend top-10 “series” using IMDB movie dataset (Link: https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows). Attach the code with output. Also, write your observation for the same.

Code

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Upload dataset
from google.colab import files
uploaded = files.upload()

# Load dataset
df = pd.read_csv(list(uploaded.keys())[0])

# Select relevant columns
df = df[['Series_Title', 'Genre', 'Overview', 'IMDB_Rating']]

# Combine features
df['content'] = df['Genre'] + " " + df['Overview']
```

```
# TF-IDF Vectorization
tfidf = TfidfVectorizer(stop_words='english')
tfidf_matrix = tfidf.fit_transform(df['content'])

# Similarity matrix
similarity = cosine_similarity(tfidf_matrix)

# Recommendation function
def recommend(title, top_n=10):
    idx = df[df['Series_Title'] == title].index[0]
    scores = list(enumerate(similarity[idx]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)

    recommendations = []
    for i in scores[1:top_n+1]:
        recommendations.append(df.iloc[i[0]]['Series_Title'])

    return recommendations

# Example
print("Top 10 Recommendations:")
print(recommend('The Shawshank Redemption'))
```

```
print(df.columns)
Index(['Series_Title', 'Genre', 'Overview', 'IMDB_Rating', 'content'], dtype='object')
```